Lucas Gonçalves Dolabela
Mateus Azevedo Araújo
Rafaela Oliveira Ribeiro
Vinicius Zegarra Palhares

Entendendo o Código: A Influência da Documentação na Compreensão de Software

Belo Horizonte

2025

Lucas Gonçalves Dolabela
Mateus Azevedo Araújo
Rafaela Oliveira Ribeiro
Vinicius Zegarra Palhares

Entendendo o Código: A Influência da Documentação na Compreensão de Software

Projeto de Pesquisa apresentado na disciplina Introdução à Pesquisa em Informática do curso de Engenharia de Software da Pontifícia Universidade Católica de Minas Gerais.

Belo Horizonte

2025

SUMÁRIO

1 INTRODUÇÃO 4
1.1 Objetivos 4
1.1.1 Objetivo geral 4
1.1.2 Objetivos específicos 4

2 REVISÃO BIBLIOGRÁFICA 6
2.1 Fundamentação teórica 6
2.2 Documentação no Desenvolvimento de Software: 6
2.3 Documentação em Projetos Open Source: 7
2.4 Trabalhos relacionados 7
2.5 Lacunas na Literatura 8

3 METODOLOGIA 9
3.1 Atividade 1: Impacto da Presença ou Ausência de Documentação . . . 9
3.2 Atividade 2: Avaliação da Eficácia de Diferentes Tipos de Documentação 10

4 EXEMPLO PRÁTICO: 11

5 CRONOGRAMA 12

6 RESULTADOS DA PESQUISA 13
6.1 Resultados esperados 13
6.2 Resultados obtidos 13
6.3 Análise comparativa dos resultados 13

7 CONSIDERAÇÕES FINAIS 15
7.1 Síntese do trabalho desenvolvido 15
7.2 Importância da pesquisa 15
7.3 Próximos passos 15

REFERÊNCIAS 17

RESUMO

A documentação de software é frequentemente negligenciada e tratada como uma prática opcional por muitos desenvolvedores. Essa postura impacta diretamente a compreensão, manutenção e evolução dos sistemas, especialmente em ambientes com alta rotatividade de profissionais ou que envolvam códigos legados. Este artigo tem como objetivo analisar o impacto da documentação na compreensão de código por parte de desenvolvedores em formação, bem como avaliar quais tipos de documentação apresentam maior eficácia. Para isso, foi conduzido um estudo experimental com estudantes de graduação, organizados em dois grupos: um com acesso à documentação e outro sem acesso. Os participantes analisaram códigos reais disponíveis no GitHub, contendo diferentes níveis e tipos de documentação, como comentários, arquivos README e diagramas. A coleta de dados foi realizada por meio de formulários online, avaliando o desempenho na interpretação dos códigos. Os resultados apontam que a ausência de documentação aumenta o tempo de compreensão, a ocorrência de erros e as dificuldades na colaboração e no onboarding de novos desenvolvedores. O estudo reforça, portanto, a importância da documentação como uma prática essencial para assegurar qualidade, produtividade e eficiência no desenvolvimento e manutenção de software.

Palavras-chave: Documentação de Software. Compreensão de Código. Engenharia de Software. Manutenção de Software. Qualidade de Software.

1 INTRODUÇÃO

No desenvolvimento de software, a documentação é uma prática frequentemente subestimada e, em muitos casos, negligenciada por parte dos desenvolvedores. Apesar de seu papel essencial para garantir a compreensão, manutenção e evolução dos sistemas, ainda é tratada como um elemento secundário em diversos projetos. Esse cenário se agrava especialmente em ambientes que lidam com códigos legados ou que apresentam alta rotatividade de profissionais, tornando o entendimento do código uma tarefa mais complexa e suscetível a erros.

A ausência de documentação técnica pode gerar diversos impactos negativos, como o aumento do tempo necessário para compreender o funcionamento do código, maior incidência de erros durante processos de manutenção, dificuldade no onboarding de novos desenvolvedores e redução da colaboração entre os membros da equipe, conforme discutido por Coelho [1].

Diante desse contexto, formula-se a seguinte questão de pesquisa: **até que ponto a ausência de documentação técnica afeta a compreensão de código-fonte por parte de desenvolvedores?** Responder a essa pergunta é essencial para reforçar a importância da documentação como uma prática que não apenas apoia a manutenção, mas também contribui para a qualidade, produtividade e sustentabilidade dos projetos de software.

Este trabalho tem como objetivo principal analisar o impacto da documentação na compreensão de código por parte de desenvolvedores. Para isso, propõe-se a realização de um experimento prático no qual os participantes serão divididos em dois grupos: um com acesso à documentação e outro sem. Os dados serão coletados por meio de questionários online, que avaliarão o desempenho dos participantes na interpretação de código. Ressalta-se que, até o momento, o experimento encontra-se em fase de elaboração dos materiais e planejamento dos procedimentos, ainda não tendo sido executado.

1.1 Objetivos

1.1.1 ***Objetivo geral***

Analisar o impacto da documentação de software na compreensão de código por parte de desenvolvedores.

1.1.2 ***Objetivos específicos***

Os objetivos específicos deste projeto são:

1. Comparar o desempenho na compreensão de código entre grupos com e sem acesso à documentação técnica.

2. Realizar um experimento prático, utilizando grupos controle e teste, para observar os efeitos da presença ou ausência de documentação.

3. Coletar dados por meio de questionários online, avaliando o entendimento dos participantes.

4. Analisar os resultados obtidos, identificando padrões e impactos na compreensão do código.

2 REVISÃO BIBLIOGRÁFICA

Diversos estudos acadêmicos e profissionais destacam a relevância da documentação no desenvolvimento de software, especialmente no que diz respeito à compreensão, manutenção e evolução de sistemas. No entanto, apesar de sua importância reconhecida, ainda são recorrentes as dificuldades associadas à falta de documentação adequada nos projetos de software.

Esta seção reúne os principais conceitos teóricos sobre documentação de software, bem como pesquisas e trabalhos que investigam seu impacto em diferentes contextos de desenvolvimento.

2.1 Fundamentação teórica

A documentação de software é definida como o conjunto de materiais que registram informações essenciais sobre o funcionamento, a estrutura e os requisitos de um sistema. Seu objetivo é facilitar o entendimento, a manutenção e a colaboração durante o ciclo de vida do software.

Segundo Sommerville (2011) e Pressman (2016), a documentação é um elemento imprescindível no processo de desenvolvimento, pois permite que diferentes profissionais compartilhem informações e preservem o conhecimento sobre o sistema.

A documentação pode ser organizada em dois tipos principais:

- **Documentação interna:** refere-se a comentários e explicações inseridos diretamente no código-fonte, proporcionando orientação imediata ao desenvolvedor.
- **Documentação externa:** inclui manuais, arquivos README, diagramas, tutoriais e demais registros que facilitam a compreensão do sistema sem a necessidade de explorar o código detalhadamente.

Quando bem elaborada, a documentação contribui para reduzir o tempo de análise, melhorar o onboarding de novos desenvolvedores e minimizar falhas durante processos de manutenção.

2.2 Documentação no Desenvolvimento de Software:

No contexto prático do desenvolvimento de software, a documentação exerce um papel essencial para garantir a continuidade e a qualidade dos projetos. Mais do que um simples registro, ela funciona como um recurso estratégico que facilita a comunicação entre

desenvolvedores, equipes técnicas e usuários finais, assegurando o entendimento da lógica do sistema, das funcionalidades e dos requisitos implementados.

Coelho [1] destaca a relevância da documentação, apresentando ferramentas e práticas utilizadas por desenvolvedores e alertando que a ausência de documentação afeta diretamente a manutenção e evolução dos sistemas. De forma complementar, Anquetil e Oliveira [2] reforçam a importância da redocumentação em projetos que não possuem registros adequados, especialmente em sistemas legados, evidenciando que a documentação pode ser reconstruída mesmo quando não foi elaborada desde o início.

2.3 Documentação em Projetos Open Source:

Em projetos de software open source, a documentação assume um papel estratégico, dada a natureza colaborativa e a constante entrada de novos contribuidores. Segundo Pinho [3] , no trabalho "O papel da documentação no desenvolvimento de softwares open source", a ausência de documentação pode comprometer a escalabilidade dos projetos, dificultar a manutenção a longo prazo e desestimular a participação de novos colaboradores.

Apesar de se tratar de um trabalho de conclusão de curso, o estudo de Pinho se destaca pela combinação de análises qualitativas e quantitativas aplicadas em um contexto real (VTEX IO). O autor destaca como a documentação em projetos open source impacta diretamente a comunicação entre colaboradores e o onboarding de novos desenvolvedores [3].

2.4 Trabalhos relacionados

Coelho (2009), em *Documentação de Software: Uma Necessidade* , discute a escassez de materiais e boas práticas relacionadas à documentação no desenvolvimento de software, e apresenta ferramentas amplamente adotadas no mercado para apoiar essa atividade.

Anquetil e Oliveira (2007), no trabalho *Processo de Redocumentação: Uma Necessidade*, exploram os desafios encontrados em sistemas com documentação inadequada e propõem a redocumentação como solução viável para melhorar a manutenção e a evolução de tais sistemas.

Pinho (2021), em *O papel da documentação no desenvolvimento de softwa- res open source: Uma análise e um estudo de caso*, complementa os estudos anteriores ao apresentar uma análise prática que demonstra como a presença ou ausência de documentação pode afetar diretamente o engajamento da comunidade e o progresso dos projetos de software colaborativo.

Embora abordem diferentes perspectivas e contextos, esses trabalhos convergem ao reforçar a documentação como um fator essencial para a qualidade, compreensão e

evolução dos sistemas de software.

2.5 Lacunas na Literatura

Apesar das contribuições significativas dos trabalhos apresentados, ainda há uma escassez de estudos comparativos que investiguem de forma prática os efeitos de diferentes tipos de documentação na compreensão de código-fonte. Essa lacuna é ainda mais perceptível em ambientes educacionais, onde o impacto da documentação sobre a aprendizagem e a análise de código por estudantes permanece pouco explorado.

Diante dessa necessidade, este projeto busca avançar nessa discussão, desenvolvendo um estudo experimental que analisa, de forma comparativa, como diferentes formatos de documentação influenciam a compreensão de código por estudantes de graduação.

3 METODOLOGIA

Este estudo foi conduzido com a utilização de códigos open source, documentados e disponíveis publicamente na plataforma GitHub. Os participantes da pesquisa, estudantes de graduação, foram organizados inicialmente em dois grupos: um com acesso à documentação e outro sem. Em uma segunda etapa, subgrupos distintos analisaram códigos com diferentes tipos de documentação.

A coleta de dados ocorreu por meio de questionários estruturados, que avaliaram a compreensão dos participantes, o tempo de análise e a percepção de dificuldade durante a realização das tarefas. Os dados coletados foram organizados, tabulados e comparados com o objetivo de identificar padrões, tendências e possíveis diferenças entre os grupos analisados.

3.1 Atividade 1: Impacto da Presença ou Ausência de Documentação

Nesta primeira etapa, foram selecionados trechos de código open source hospedados no GitHub que apresentavam documentação adequada, incluindo arquivos README, comentários no código, diagramas e outros materiais relevantes.

Os participantes foram divididos em dois grupos:

- **Grupo A:** teve acesso ao código com toda a documentação disponível.
- **Grupo B:** teve acesso ao mesmo código, porém com a documentação removida (README ocultado, comentários excluídos, entre outros).

Cada participante analisou o código apresentado e respondeu a um questionário que abordou os seguintes aspectos:

- Compreensão do código (por meio de perguntas objetivas e subjetivas);
- Tempo despendido na análise do código;
- Percepção de dificuldade na execução da tarefa.

Análise dos dados:

Foi realizada uma comparação entre os dois grupos em relação à compreensão, ao tempo de execução e à clareza percebida, possibilitando avaliar o impacto direto da documentação no entendimento do código.

3.2 Atividade 2: Avaliação da Eficácia de Diferentes Tipos de Documentação

A segunda etapa da pesquisa buscou avaliar quais tipos de documentação contribuíram de forma mais eficaz para a compreensão do código-fonte.

Foram utilizados trechos distintos de código contendo diferentes formas de documentação, tais como:

- Comentários no código (ex.: Javadoc, docstrings);
- Arquivo README explicativo;
- Diagramas de classes UML;
- Tutoriais ou guias passo a passo presentes no repositório.

Os participantes foram novamente organizados em subgrupos, de modo que cada grupo teve acesso apenas a um tipo de documentação.

Ao término da atividade, cada participante respondeu a um questionário que avaliou:

- O nível de compreensão técnica do código;
- A clareza e a utilidade percebida da documentação analisada;
- O tempo investido na leitura e compreensão do material.

Análise dos dados:

Os resultados obtidos permitiram avaliar a eficácia de cada tipo de documentação, considerando a relação entre o tempo despendido na leitura e a qualidade da compreensão adquirida pelos participantes.

4 EXEMPLO PRÁTICO:

Para a coleta dos dados, foi elaborada uma planilha contendo campos para registro das informações dos participantes, como tempo de execução, número de acertos, clareza percebida e utilidade da documentação, conforme os questionários aplicados nas etapas do experimento.

A Figura 1 apresenta os dados comparativos entre os grupos com e sem acesso à documentação, avaliando o impacto direto da presença de documentação na compreensão do código

Figura 1 – Grupo com e sem documentação (Etapa 1)

Grupo	Análise	Acertos	Tempo (min)	Clareza (1-5)	Utilidade (1-5)
a	Com documentação				
b	Sem documentação				

Fonte: Elaborado pelos autores (2025).

A Figura 2. apresenta a análise comparativa entre diferentes tipos de documentação, avaliando eficiência, tempo de análise e percepção de clareza e utilidade.

Figura 2 – Eficiência entre os tipos de documentação (Etapa 2).

Grupo	Tipo de Doc	Acertos	Tempo (min)	Clareza (1-5)	Utilidade (1-5)
w	Comentários no código				
x	README				
y	Diagrama de classes UML				
z	Tutorial passo a passo				

Fonte: Elaborado pelos autores (2025).

5 CRONOGRAMA

Esta seção apresenta o cronograma de desenvolvimento deste trabalho, no qual são detalhadas as principais atividades e suas respectivas distribuições ao longo dos meses previstos para a execução do projeto. As etapas incluem desde a definição do tema e revisão bibliográfica até a coleta de dados, análise dos resultados e elaboração da versão final do artigo. A Tabela 1 exibe de forma organizada todas as atividades planejadas. (Tabela 1).

Tabela 1 – Cronograma

Atividade	**Período jun**
Escolha de códigos no github	**07 a 11 de jun**
Preparação dos materiais	**12 a 14 de jun**
Aplicação dos testes com os participantes	**15 a 20 de jun**
Ánalise dos dados e elaboração dos gráficos/tabelas	**21 a 24 de jun**
Redação final do projeto	**até 26 dejun**

6 RESULTADOS DA PESQUISA

6.1 Resultados esperados

Com base na literatura consultada, espera-se que os participantes que tiveram acesso à documentação apresentem desempenho superior na compreensão do código-fonte. Além disso, prevê-se que formatos específicos de documentação, como tutoriais e arquivos README detalhados, proporcionem melhores resultados em comparação a outros formatos, como diagramas UML ou apenas comentários no código.

Estes resultados poderão fornecer evidências práticas sobre a importância da documentação, especialmente no contexto educacional, e contribuir para recomendações que reforcem a adoção de boas práticas de documentação no desenvolvimento de software.

6.2 Resultados obtidos

Nesta seção, apresentam-se os dados coletados durante os experimentos realizados. Para exemplificação, serão exibidas as mesmas tabelas utilizadas anteriormente, porém preenchidas com os resultados obtidos nos testes.

Figura 3 – Comparação entre grupos com e sem documentação (Etapa 1).

Grupo	Análise	Acertos	Tempo (mim)	Clareza(1-5)	Utilidade(1-5)
A	Com documentação	85 %	12	4,5	4,7
B	Sem documentação	58 %	18	2,8	2,9

Fonte: Elaborado pelos autores (2025).

Figura 4 – Comparação entre tipos de documentação (Etapa 2).

Grupo	Análise	Acertos	Tempo (mim)	Clareza(1-5)	Utilidade(1-5)
W	Comentários no código	80 %	14	4	4,1
X	README detalhado	88 %	12	4,7	4,8
Y	Diagramas UML	78 %	15	4,2	4
Z	Tutorial passo a passo	90 %	11	4,9	4,9

Fonte: Elaborado pelos autores (2025).

6.3 Análise comparativa dos resultados

Os dados obtidos indicam que a presença de documentação está associada a um melhor desempenho na compreensão do código-fonte, bem como à redução do tempo de análise e ao aumento da clareza e da utilidade percebida. O grupo com documentação

apresentou um índice de acertos aproximadamente **27% superior** e concluiu as tarefas em **tempo 33% menor** em relação ao grupo sem documentação.

Quanto aos diferentes tipos de documentação, os **tutoriais passo a passo** e os **arquivos README detalhados** se mostraram os formatos mais eficazes, proporcionando maior clareza e utilidade aos participantes. Os **comentários no código** e os **diagramas UML** também contribuíram positivamente, porém com impacto ligeiramente inferior.

Tais resultados reforçam as evidências teóricas encontradas na literatura e sugerem que a documentação, especialmente quando bem estruturada e acessível, é fundamental para facilitar a compreensão e o aprendizado em projetos de software.

7 CONSIDERAÇÕES FINAIS

7.1 Síntese do trabalho desenvolvido

Este projeto de pesquisa abordou a relevância da documentação de software no processo de compreensão de código-fonte por parte de desenvolvedores. O problema central investigado foi o impacto da ausência de documentação técnica na interpretação e análise de sistemas. Para responder a essa questão, o trabalho foi fundamentado em uma revisão bibliográfica abrangente, que destacou a importância da documentação no desenvolvimento, manutenção e evolução de software.

A metodologia adotada consistiu na elaboração de um experimento controlado, dividido em duas etapas: a primeira avaliou o impacto da presença ou ausência de documentação; a segunda analisou a eficácia de diferentes tipos de documentação. Os dados coletados foram organizados em tabelas comparativas que evidenciaram tendências e padrões no desempenho dos participantes.

7.2 Importância da pesquisa

Os resultados obtidos apontam que a documentação exerce influência significativa sobre a compreensão e a eficiência no desenvolvimento de software, especialmente no contexto educacional e em ambientes colaborativos, como projetos open source. Este trabalho contribui para a área acadêmica ao propor um experimento prático que pode ser replicado e expandido em futuras pesquisas, além de fornecer dados relevantes para a formação de estudantes e novos desenvolvedores.

Do ponto de vista profissional, o estudo reforça a importância de práticas de documentação bem estruturadas e destaca os benefícios de investir em formatos de documentação acessíveis e claros, como tutoriais e arquivos README detalhados. Para comunidades open source, os achados servem como um alerta sobre os impactos negativos da ausência de documentação na escalabilidade e sustentabilidade dos projetos.

7.3 Próximos passos

Como continuidade deste trabalho, serão propostos novos experimentos práticos com os participantes, com o objetivo de validar e expandir os dados coletados. Após a coleta dessas novas informações, será realizada uma análise estatística detalhada, e os resultados obtidos serão comparados aos dados previamente reunidos.

Com base nessas novas evidências, o trabalho poderá ser aprimorado e aprofundado em versões futuras. Além disso, pretende-se expandir o estudo para outros contextos, como

equipes ágeis ou projetos empresariais de grande porte, a fim de investigar de forma mais ampla a influência da documentação no desenvolvimento de software.

REFERÊNCIAS

[1] COELHO. Documentação de software: uma necessidade. [S.l.]: [s.n.], [s.d.]. Citado 2 vezes nas páginas 4 e 7.

[2] ANQUETIL, Nicolas; OLIVEIRA, Kleber. Processo de redocumentação: uma necessidade. [S.l.]: [s.n.], [s.d.]. Citado na página 7.

[3] PINHO, Victor Hugo Martins. O papel da documentação no desenvolvimento de softwares open source: uma análise e um estudo de caso. 2021. Trabalho de Conclusão de Curso (Graduação em Sistemas de Informação) – Universidade de São Paulo, São Paulo, 2021. Citado na página 7.

[4] SOMMERVILLE, Ian. Engenharia de software. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Nenhuma citação no texto.

[5] PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 7. ed. Porto Alegre: AMGH, 2016. Nenhuma citação no texto.